OS Practical No. 7

Aim :

Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

Code :

```
devanshu — nano bankers.c — 119×69

  UW PICO 5.09                                        File: bankers.c

#include <stdio.h>

#define P 5   // Number of processes
#define R 4   // Number of resource types

int main() {
    int allocation[P][R], max[P][R], need[P][R];
    int available[R];
    int finish[P] = {0};
    int safeSequence[P];
    int work[R];
    int i, j, count = 0;

    printf("Enter Allocation Matrix (5 processes x 4 resources):\n");
    for(i = 0; i < P; i++) {
        for(j = 0; j < R; j++) {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("\nEnter Max Matrix (5 processes x 4 resources):\n");
    for(i = 0; i < P; i++) {
        for(j = 0; j < R; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    printf("\nEnter Available Resources (4 values):\n");
    for(i = 0; i < R; i++) {
        scanf("%d", &available[i]);
        work[i] = available[i];
    }

    // Calculate Need matrix
    for(i = 0; i < P; i++) {
        for(j = 0; j < R; j++) {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }

    printf("\nNeed Matrix:\n");
    for(i = 0; i < P; i++) {
        for(j = 0; j < R; j++) {
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }

    // Banker's Algorithm
    while(count < P) {
        int found = 0;

        for(i = 0; i < P; i++) {
            if(finish[i] == 0) {
                int possible = 1;

                for(j = 0; j < R; j++) {
                    if(need[i][j] > work[j]) {
                        possible = 0;
                        break;
                    }
                }

                if(possible) {

^G Get Help     ^O WriteOut     ^R Read File    ^Y Prev Pg      ^K Cut Text     ^C Cur Pos
^X Exit         ^J Justify      ^W Where is     ^V Next Pg      ^U UnCut Text   ^T To Spell
```

```
#include <stdio.h>

#define P 5   // Number of processes
```

```
#define R 4   // Number of resource types

int main() {
    int allocation[P][R], max[P][R], need[P][R];
    int available[R];
    int finish[P] = {0};
    int safeSequence[P];
    int work[R];
    int i, j, count = 0;

    printf("Enter Allocation Matrix (5 processes x 4 resources):\n");
    for(i = 0; i < P; i++) {
        for(j = 0; j < R; j++) {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("\nEnter Max Matrix (5 processes x 4 resources):\n");
    for(i = 0; i < P; i++) {
        for(j = 0; j < R; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    printf("\nEnter Available Resources (4 values):\n");
    for(i = 0; i < R; i++) {
        scanf("%d", &available[i]);
        work[i] = available[i];
    }

    // Calculate Need matrix
    for(i = 0; i < P; i++) {
        for(j = 0; j < R; j++) {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }

    printf("\nNeed Matrix:\n");
    for(i = 0; i < P; i++) {
        for(j = 0; j < R; j++) {
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }

    // Banker's Algorithm
    while(count < P) {
        int found = 0;

        for(i = 0; i < P; i++) {
            if(finish[i] == 0) {
                int possible = 1;

                for(j = 0; j < R; j++) {
                    if(need[i][j] > work[j]) {
                        possible = 0;
                        break;
                    }
                }

                if(possible) {
                    for(j = 0; j < R; j++) {
```

```
                work[j] += allocation[i][j];
            }

            safeSequence[count++] = i;
            finish[i] = 1;
            found = 1;
        }
    }
}

    if(found == 0) {
        break;
    }
}

if(count == P) {
    printf("\nSystem is in SAFE state.\nSafe Sequence: ");
    for(i = 0; i < P; i++) {
        printf("P%d ", safeSequence[i]);
    }
    printf("\n");
} else {
    printf("\nSystem is NOT in safe state.\n");
}

return 0;
}
```

OUTPUT:

```
devanshu — -zsh — 119×69

Last login: Fri Feb 27 16:42:47 on ttys000
devanshu@Devanshus-MacBook-Air ~ % gcc bankers.c -o bankners_program
devanshu@Devanshus-MacBook-Air ~ % ./bankers_program
Enter Allocation Matrix (5 processes x 4 resources):
0 0 1 2
1 0 0 0
1 3 5 4
0 6 3 2
0 0 1 4

Enter Max Matrix (5 processes x 4 resources):
0 0 1 2
1 7 5 0
2 3 5 6
0 6 5 2
0 6 5 6

Enter Available Resources (4 values):
1 5 2 0

Need Matrix:
0 0 0 0
0 7 5 0
1 0 0 2
0 0 2 0
0 6 4 2

System is in SAFE state.
Safe Sequence: P0 P2 P3 P4 P1
devanshu@Devanshus-MacBook-Air ~ %
```